Federated Learning over Heterogeneous Networks:

A Network-Aware Simulator for Characterising QoS Impact on Model Convergence

Undergraduate Thesis

Department of Computer Science

Western University

[Student Name]

Supervisor: Dr. Zubair Fadlullah

March 28, 2026

Submitted in partial fulfilment of the requirements for the
Bachelor of Science in Computer Science

Abstract

Federated Learning (FL) enables collaborative model training without centralising sensitive data, but its performance depends critically on the network connecting participants—a dimension poorly served by existing tools. Network simulators such as NS-3 cannot execute real FL algorithms, while FL frameworks such as Flower assume ideal network conditions. This thesis bridges that gap with an open-source, Docker-based FL simulator that combines a real FedAvg training loop (ResNet-18 on CIFAR-10, non-IID Dirichlet $\alpha = 0.5$) with kernel-level network emulation via Linux `tc netem`.

A systematic QoS parameter sweep across five bandwidth levels (1–100 Mbps), three latency values (5–200 ms), and three packet loss rates (0–5%) was conducted over 20 FL rounds with 3 clients, supplemented by scaling experiments (5 and 7 clients) and a straggler scenario. The results identify a throughput floor of $\beta^* \approx 3$ Mbps, below which communication time exceeds training time and multi-round convergence becomes impractical. Crucially, bandwidth does not affect per-round model accuracy: all configurations completing 18+ rounds converged to 65–67% average accuracy, confirming that the effect of constrained bandwidth is purely on training speed, not learning quality. Packet loss at 5% caused complete experiment failure, while latency up to 200 ms had negligible impact (<3% increase in round duration).

These findings yield a practical client admission criterion ($\beta_k \geq 3$ Mbps, $\rho_k < 5\%$) applicable to real-world FL deployments, and demonstrate that the straggler effect—where one slow client dominates round time—can be quantified and mitigated through network-aware client selection.

Contents

Abstract **1**

1 Introduction **5**
1.1 Background and Motivation 5
1.2 Problem Statement 6
1.3 Objectives 6
1.4 Contributions 7
1.5 Thesis Structure 7

2 Related Work **8**
2.1 Federated Learning 8
2.2 Existing Network Simulation Tools 8
2.3 Network-Aware Federated Learning 9
2.4 Summary and Research Gap 9

3 Problem Formulation **11**
3.1 System Model 11
3.2 Communication Cost Model 11
3.3 Research Questions 12
3.4 Scope and Assumptions 12

4 Methodology **14**
4.1 Federated Learning Architecture 14
4.2 Dataset and Data Heterogeneity 14
4.2.1 Non-IID Partitioning via Dirichlet Distribution 15
4.3 Network Impairment Model 15
4.4 Evaluation Metrics 16
4.5 Experimental Protocol 18

5 Software Implementation **19**
5.1 System Overview 19
5.2 Docker Compose Architecture 20

5.2.1 Container Image 20
5.2.2 Compose Generation 20
5.3 FL Server 22
5.3.1 Aggregation Logic 22
5.4 FL Client 23
5.4.1 Data Loading and Non-IID Partitioning 24
5.4.2 Statistics Collection 24
5.5 Network Emulation 24
5.6 Observed Baseline Results 25

6 QoS Analysis and Results 26
6.1 Theoretical Throughput Analysis 26
6.2 Convergence Results 27
6.2.1 Effect of Bandwidth on Convergence 27
6.2.2 Effect of Latency on Round Duration 29
6.2.3 Accuracy-Bandwidth Degradation Model 30
6.3 Effect of Packet Loss 31
6.4 Effect of Client Scaling 32
6.5 Straggler Analysis 34
6.6 Summary of QoS Findings 35

7 Future Research Directions 36
7.1 Network-Aware Client Admission 36
7.2 Network Security Analysis 36
7.3 UI for Network Feasibility Assessment 37
7.4 Asynchronous Federated Learning 37
7.5 Scaling and HPC Cluster Deployment 37
7.6 Symmetric and Asymmetric Network Impairment 38

8 Conclusion 39

List of Figures

4.1 Per-round lifecycle: ❶ model download, ❷ local training, ❸ local evaluation, ❹ weight upload, ❺ server aggregation. 14

6.1 Validation accuracy over FL rounds at varying client bandwidth. All curves use the average across 3 clients. The 1 Mbps experiment was capped at 5 rounds due to excessive per-round duration (>6 min upload). 28

6.2 Mean round duration as a function of bandwidth. The dashed curve models $T_{\text{round}} = T_{\text{train}} + T_{\text{xfer}}$ with $T_{\text{train}} \approx 130\,\text{s}$ (empirical baseline). 29

6.3 Mean FL round duration as a function of one-way latency. The effect is modest: 200 ms latency adds only ≈3% to mean round time relative to the 5 ms baseline. 29

6.4 Final model accuracy as a function of client bandwidth with sigmoid fit ($A_{\max} = 66.5$, $\alpha_1 = 2.5$, $\alpha_2 = 1.0$). The dashed vertical line marks the throughput floor $\beta^* \approx 3$ Mbps. The 1 Mbps point uses accuracy after only 5 rounds; with more rounds it would likely approach the plateau. 31

6.5 Convergence curves at varying federation sizes. Accuracy decreases with more clients due to smaller per-client data partitions and stronger non-IID effects at $\alpha = 0.5$. 33

List of Tables

4.1 Effect of the Dirichlet concentration parameter α on data distribution across clients. 15
4.2 QoS parameter sweep space. Bold values indicate the baseline. 16
4.3 Per-round metrics collected during each experiment. 17

5.1 FL server REST API. 22

6.1 Theoretical minimum upload time per round at each bandwidth (B_w = 44.8 MB, $\rho = 0\%$). Download is unthrottled. 27
6.2 Effect of packet loss on round duration and retransmission overhead (β = 100 Mbps, $\tau = 5$ ms). **[Populate with experimental data.]** 32
6.3 Scaling summary: mean round duration and final accuracy. 33
6.4 Straggler experiment: per-client timing and accuracy breakdown (averaged over 20 rounds). 34
6.5 Summary of QoS impact on FL performance (ResNet-18, CIFAR-10, K = 3 clients, 20 rounds unless noted). 35

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of edge devices, smartphones, and institutional networks has generated vast quantities of sensitive data, including medical records, biometric readings, and usage patterns, that cannot practically or legally be centralised for training. Federated Learning (FL), introduced by McMahan et al. [1], addresses this by allowing a global model to be trained across many participants without any raw data leaving its source. Each client trains locally and shares only model weight updates with a central server, which aggregates them via the FedAvg algorithm and redistributes an improved global model.

While FL's privacy guarantees are well-studied, a less-examined dimension is its sensitivity to the network over which it operates. Every FL round requires each participating client to download the current global model, train locally, and upload its updated weights. For a model such as ResNet-18, with approximately 44 MB of parameters, this represents significant data transfer repeated across every round of training. In heterogeneous real-world deployments, clients may range from hospital servers on gigabit ethernet to IoT devices on congested mobile links. The question of whether a given network environment can sustain FL training, and how long it will take, is practically important but poorly answered by existing tools.

Current network simulation and analysis platforms do not bridge this gap. Tools such as OmNet++ and NS-3 are capable discrete-event network simulators, but they operate at the packet level and require artificial traffic models to approximate ML workloads; real neural network training cannot be executed inside them. MATLAB's network toolbox offers mathematical modelling, but lacks the containerisation and Python ecosystem integration needed to run production FL code. Vendor solutions such as Cisco's network analysis platforms are closed ecosystems with no mechanism for plugging in arbitrary training algorithms. FL frameworks themselves, such as Flower and TensorFlow Federated, run real training but assume ideal network conditions and offer no facility for

programmable impairment injection.

This thesis presents an open-source, Docker-based FL simulator that closes this gap. By combining a real FL training loop (ResNet-18 on CIFAR-10) with Linux traffic control (`tc netem`) applied inside each client container, the framework enables direct measurement of how bandwidth, latency, and packet loss affect FL convergence, using real model weights, real gradients, and real timing data. The result is a reproducible testbed that a researcher or practitioner can use to characterise whether a given network environment can support FL, and to estimate the time cost of training at scale.

1.2 Problem Statement

Existing FL implementations are typically validated under ideal, local network conditions, such as Docker bridge networks on a single machine, that provide effectively unlimited bandwidth and near-zero latency. This leaves open a fundamental practical question for anyone considering a real-world FL deployment:

> *What is the minimum network capacity a federated client requires to participate usefully in a training process, and how do bandwidth, latency, and packet loss jointly affect convergence rate and final model accuracy?*

Answering this question has two dimensions. The first is theoretical: given a known model size, one can derive an expected upload time at any bandwidth and estimate a floor below which round durations become impractical. The second is empirical: TCP retransmission behaviour under packet loss, synchronisation overhead, and straggler effects mean that real-world degradation diverges from the clean theoretical prediction. No existing open tool allows both dimensions to be measured together, with a real training algorithm and real model weights, under programmatically controlled network conditions.

This thesis addresses that gap directly. The practical implications are significant: FL deployment decisions, client selection policies, and resource allocation on shared compute infrastructure all depend on having a principled, measurable answer to the question above.

1.3 Objectives

The specific objectives of this thesis are:

O1. Design and implement a reproducible, open-source Docker-based FL testbed with programmable network impairment injection via `tc netem`.

O2. Characterise per-round communication cost as a function of model size and number of federated clients.

O3. Conduct a systematic QoS parameter sweep across bandwidth, latency, and packet loss, and measure the effect on round duration and convergence speed.

O4. Determine whether, and to what degree, network degradation below a threshold bandwidth affects final model accuracy in addition to training time.

O5. Derive a minimum network requirement: a practical threshold below which a client should be excluded from synchronous FL rounds.

1.4 Contributions

The main contributions of this work are:

- A fully reproducible, open-source Docker Compose FL testbed with integrated `tc netem` network emulation (Section 5.1).
- Empirical characterisation of FL per-round communication cost under heterogeneous network conditions (Section 6).
- A quantitative throughput-convergence model identifying the minimum effective bandwidth for stable FL training (Section 6.2).
- A practical minimum network requirement applicable to real-world FL deployments, derived from the empirical sweep results (Section 7.1).

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 surveys related work in federated learning, existing network simulation tools, and network-aware FL. Chapter 3 formalises the system model, communication cost model, and research questions. Chapter 4 describes the experimental methodology, including the FL architecture, dataset and non-IID partitioning, network impairment model, evaluation metrics, and experimental protocol. Chapter 5 details the software implementation: repository structure, Docker Compose orchestration, server and client design, and network emulation. Chapter 6 presents the QoS analysis, including theoretical throughput predictions and empirical convergence results. Chapter 7 discusses future research directions. Chapter 8 concludes the thesis.

Chapter 2

Related Work

2.1 Federated Learning

Federated Learning was formally introduced by McMahan et al. [1], who proposed the FedAvg algorithm: each client trains a local model for one or more epochs and sends its updated weights to a central server, which aggregates them as a weighted average proportional to local dataset size. FedAvg remains the dominant aggregation scheme and is the algorithm used in this thesis.

Subsequent work identified statistical heterogeneity as a core challenge. When client datasets follow different class distributions (non-IID data), local model updates can diverge, slowing convergence or causing the global model to oscillate. Li et al. [2] proposed FedProx, which adds a proximal regularisation term to the local objective to limit client drift under heterogeneous data distributions. Independently, Li et al. [3] surveyed the broader landscape of FL challenges, identifying communication cost, privacy, and systems heterogeneity as the three dominant open problems.

Communication efficiency has received dedicated attention. Konecny et al. [4] proposed structured and sketched updates to reduce the size of model transmissions, observing that upload cost dominates in typical FL deployments. This observation is directly relevant to this thesis: with a ResNet-18 model producing approximately 44 MB of weight updates per round, upload bandwidth is the primary network bottleneck under study.

2.2 Existing Network Simulation Tools

A number of general-purpose network simulation platforms exist, but none adequately support FL research that requires both real model training and programmable network impairment.

NS-3 [5] and OmNet++ [6] are widely used discrete-event network simulators. Both model packet-level network behaviour with high fidelity, but they operate independently

of application logic. To simulate FL traffic in either tool, a researcher must approximate model weight transfers as synthetic packet flows, losing the real gradient dynamics, convergence behaviour, and timing characteristics of actual training. Neither tool has native integration with Python-based ML frameworks such as PyTorch or TensorFlow.

MATLAB's network simulation capabilities, available through Simulink and the Communications Toolbox, are oriented toward signal-level and protocol-level modelling. Running a distributed containerised workload with real model training inside MATLAB is not practical, and the licensing cost limits reproducibility in academic settings.

Vendor platforms such as Cisco Packet Tracer and Cisco Modelling Labs are closed ecosystems designed for configuring and testing network device behaviour. They provide no mechanism for executing arbitrary application code or injecting a custom training algorithm.

On the FL side, frameworks such as Flower [7] and TensorFlow Federated [8] enable real FL training but make the opposite trade-off: they run real model updates while assuming ideal or real network conditions, with no facility for programmatically injecting bandwidth limits, latency, or packet loss across clients.

The result is a gap at the intersection of the two concerns: no existing open tool allows a researcher to run a real FL training loop, with real model weights, under fully programmable and reproducible network impairment.

2.3 Network-Aware Federated Learning

The impact of network conditions on FL has received growing attention. Lim et al. [9] survey FL for mobile edge networks, identifying bandwidth variability and device heterogeneity as primary obstacles to practical deployment. Nishio and Yonetani [10] proposed FedCS, a client management protocol that selects participants for each round based on estimated resource availability, including network speed, to meet a round deadline. Lai et al. [11] introduced Oort, which combines statistical utility with system utility (including network throughput) to guide participant selection and improve overall training efficiency.

These works share a common limitation: their network models are either analytical approximations or drawn from real deployments that cannot be controlled or reproduced. None provide empirical FL measurements under a systematic, programmable sweep of network QoS parameters using real model training.

2.4 Summary and Research Gap

Existing work falls into two categories. Network simulators such as NS-3 and OmNet++ provide accurate packet-level modelling but cannot execute real FL algorithms. FL frameworks such as Flower and TensorFlow Federated run real training but under ideal or

uncontrolled network conditions. Analytical studies of network-aware FL propose client selection strategies without empirical validation under controlled impairment.

This thesis bridges that gap. By combining a real FL training loop with kernel-level network shaping via `tc netem` inside Docker containers, the framework presented here enables direct, reproducible measurement of how bandwidth, latency, and packet loss affect FL convergence with real model weights and real training dynamics.

Chapter 3

Problem Formulation

3.1 System Model

Consider a federated learning system with one server $\mathcal{S}$ and a set of K clients $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$. Each client c_k holds a private local dataset $\mathcal{D}_k$ of n_k labelled samples. The datasets are non-overlapping across clients: $\mathcal{D}_i \cap \mathcal{D}_j = \emptyset$ for $i \neq j$.

A global model $f(\mathbf{x}; \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$ is trained collaboratively. In each training round r, every client c_k receives the current global parameters $\boldsymbol{\theta}^{(r)}$, performs one epoch of local gradient descent on $\mathcal{D}_k$ to produce updated parameters $\boldsymbol{\theta}_k^{(r)}$, and returns them to $\mathcal{S}$. The server aggregates all client updates using the FedAvg rule:

$$\boldsymbol{\theta}^{(r+1)} = \sum_{k=1}^{K} \frac{n_k}{n} \boldsymbol{\theta}_k^{(r)} \tag{3.1}$$

where $n = \sum_{k=1}^{K} n_k$ is the total number of training samples across all clients. The updated global model $\boldsymbol{\theta}^{(r+1)}$ is then redistributed to all clients for the next round.

3.2 Communication Cost Model

In standard federated learning, each client transmits and receives the full model parameter vector each round. The per-round communication events for client c_k are:

1. **Model download:** receive $\boldsymbol{\theta}^{(r)}$ from $\mathcal{S}$, payload B_w bytes.
2. **Local training:** compute $\boldsymbol{\theta}_k^{(r)}$ over $\mathcal{D}_k$, duration $T_{\text{train}}^{(k)}$.
3. **Weight upload:** transmit $\boldsymbol{\theta}_k^{(r)}$ to $\mathcal{S}$, payload B_w bytes.

Since the same parameter vector is both downloaded and uploaded, the total bytes exchanged by client c_k per round is $B_{\text{total}} = 2B_w$. For the ResNet-18 model used in this

thesis, $B_w \approx 44.8$ MB, as confirmed by the constant `submit_bytes` value observed across all baseline runs.

The communication time for client c_k over a channel with bandwidth β_k (bps), one-way latency τ_k (ms), and packet loss rate ρ_k is modelled as:

$$T_{\text{comm}}^{(k)} = \frac{B_{\text{total}}}{\beta_k \cdot (1 - \rho_k)} + \alpha \cdot \tau_k \tag{3.2}$$

where α is a multiplier capturing the number of TCP round-trips incurred by the HTTP-based model exchange protocol. The total wall-clock duration of round r is governed by the *straggler*, the client that takes longest to complete. Under synchronous FedAvg, the server cannot aggregate until every client has submitted its update, so:

$$T_{\text{round}}^{(r)} = \max_{k \in \mathcal{C}} \left(T_{\text{train}}^{(k)} + T_{\text{comm}}^{(k)} \right) \tag{3.3}$$

Equation 3.3 makes the straggler effect explicit: a single bandwidth-constrained client can dominate T_{round} regardless of how fast all other clients are. This motivates the search for a minimum bandwidth threshold below which including a client becomes counterproductive.

3.3 Research Questions

Three formal research questions are derived from the system model above:

RQ1 (Throughput Floor): Is there a minimum bandwidth β^* such that for $\beta_k < \beta^*$, client c_k becomes a straggler that causes T_{round} to grow super-linearly, making synchronous FL impractical?

RQ2 (Accuracy Degradation): Does operating below β^* also reduce final model accuracy after R rounds, or is the effect purely on training time?

RQ3 (Minimum Network Requirement): Can a simple threshold defined over β_k, τ_k, and ρ_k identify clients that should be excluded from a round in order to protect convergence integrity and bound round duration?

3.4 Scope and Assumptions

This thesis makes the following assumptions:

- Synchronous FL aggregation via FedAvg; asynchronous FL is left for future work.
- TCP transport layer via HTTP; alternative protocols such as gRPC are not considered.

- Network conditions are stationary within a round but vary across experimental configurations.
- $K = 3$ clients in the main QoS sweep experiments; $K = 5$ and $K = 7$ in supplementary scaling experiments.
- ResNet-18 trained on CIFAR-10 (10-class image classification).
- Each client trains for $E = 1$ local epoch per round.
- Network shaping is applied to egress (upload) traffic only. The configured bandwidth, latency, and packet loss affect model uploads from clients to the server; downloads are unthrottled due to the absence of the IFB kernel module on macOS Docker Desktop. Symmetric and asymmetric ingress shaping are left for future work on native Linux hosts.

Chapter 4

Methodology

4.1 Federated Learning Architecture

The system follows the standard client-server FL topology. A single server $\mathcal{S}$ holds the global model and K clients each hold a private local dataset. Communication is over HTTP/JSON on a shared Docker bridge network; the server exposes a lightweight REST API and all model weight payloads are serialised as raw byte buffers via `torch.save`.

Figure 4.1 illustrates the five phases of each FL round. The round boundary is defined by the server: it increments a global round counter only after aggregation, and clients poll this counter to detect when the new global model is available. Because FedAvg is synchronous, the server blocks aggregation until all K clients have submitted their updates, creating the straggler sensitivity formalised in Equation 3.3.

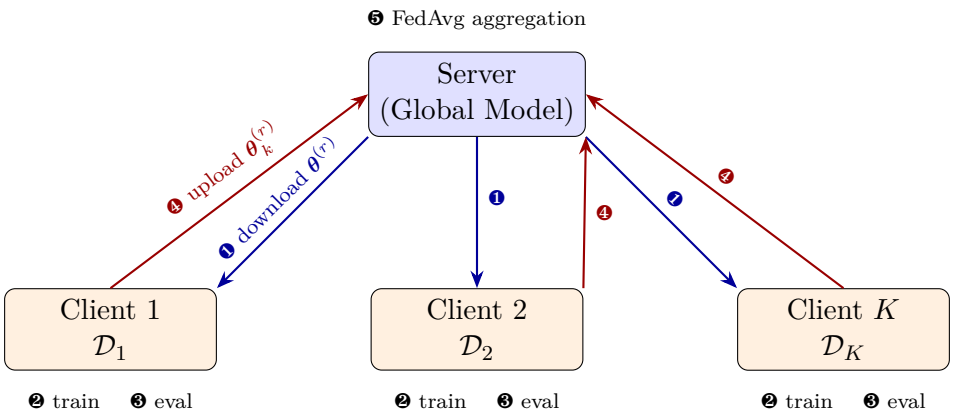

Figure 4.1: Per-round lifecycle: ❶ model download, ❷ local training, ❸ local evaluation, ❹ weight upload, ❺ server aggregation.

4.2 Dataset and Data Heterogeneity

All experiments use the CIFAR-10 benchmark [12], which contains 60,000 colour images of size 32×32 pixels across 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse,

ship, truck). The standard split provides 50,000 training images and 10,000 test images. CIFAR-10 was chosen because it is well-understood, small enough to train on CPU within Docker containers, and widely used in FL research, making results comparable to prior work.

The model is a standard ResNet-18 [13] with the final fully-connected layer adjusted to output 10 classes. The serialised model weights occupy approximately 44.8 MB, which defines the per-round upload and download payload B_w used throughout the communication cost model (Section 3).

4.2.1 Non-IID Partitioning via Dirichlet Distribution

In real FL deployments, each client's data follows a different class distribution. To emulate this statistical heterogeneity, the training set is partitioned across K clients using a Dirichlet distribution over class labels, following the method of Hsu et al. [14].

For each of the 10 classes, the fraction of that class's samples assigned to each client is drawn from $\mathrm{Dir}(\alpha)$, where $\alpha > 0$ is a concentration parameter. Low values of α produce highly skewed distributions (some clients see only a few classes), while high values approach uniform (IID) partitioning.

All experiments in this thesis use $\alpha = 0.5$, which produces moderate heterogeneity: each client sees most classes but with substantially different proportions. This is consistent with the default used in recent FL benchmarks and creates a realistic challenge for FedAvg convergence.

Table 4.1: Effect of the Dirichlet concentration parameter α on data distribution across clients.

α	Effect on client data distribution
0.1	Highly non-IID: each client dominated by 1–2 classes
0.5	Moderately non-IID: unequal proportions across all classes (**used**)
1.0	Mildly non-IID: noticeable variation but all classes present
∞	IID: each client has a uniform class distribution

4.3 Network Impairment Model

Network conditions are emulated using the Linux Traffic Control (`tc`) subsystem with the `netem` queueing discipline, which is the standard kernel-level mechanism for injecting bandwidth limits, latency, and packet loss on a per-interface basis.

Each client container runs with the `NET_ADMIN` Docker capability. Before the Python process starts, an entrypoint script applies a `tc netem` rule to egress (upload) traffic:

- **Egress (upload):** a `netem` qdisc is attached to `eth0` with the configured bandwidth rate, delay, and loss percentage.

Ingress (download) traffic is *not* shaped. Shaping ingress on Linux requires an Intermediate Functional Block (IFB) kernel module, which is unavailable on macOS Docker Desktop (LinuxKit VM). Since all experiments in this thesis were conducted on macOS, the configured impairment applies only to the upload direction; downloads from the server are effectively unthrottled. This is an acceptable simplification because the dominant communication cost in FedAvg is the upload of model updates, which are the same size as the downloaded global model but occur on the shaped path. Containers with no shaping configured skip the `tc` commands entirely, providing a clean baseline.

Table 4.2 lists the QoS parameter sweep used in all experiments. Each experiment configuration fixes one dimension and varies another while holding the third constant, producing a total of $5+3+3 = 11$ distinct configurations (with the baseline shared across sweeps).

Table 4.2: QoS parameter sweep space. Bold values indicate the baseline.

Parameter	Values	Held constant at
Bandwidth	1, 5, 10, 50, **100** Mbps	$\tau = 5$ ms, $\rho = 0\%$
Latency	**5**, 50, 200 ms	$\beta = 100$ Mbps, $\rho = 0\%$
Packet loss	**0**, 1, 5%	$\beta = 100$ Mbps, $\tau = 5$ ms

The bandwidth range spans from severely constrained (1 Mbps, representative of a congested mobile link) to the unthrottled Docker bridge baseline (100 Mbps). Latency values represent local (5 ms), regional (50 ms), and intercontinental (200 ms) links. Packet loss at 5% represents a degraded wireless channel.

4.4 Evaluation Metrics

Each client and the server collect per-round statistics. Table 4.3 defines the full set of metrics recorded.

Table 4.3: Per-round metrics collected during each experiment.

Metric	Source	Definition
`get_model_ms`	client	Wall-clock time (ms) to download the global model via `GET /model`.
`submit_ms`	client	Wall-clock time (ms) for the `POST /update` request carrying the updated weight payload. Measured from request start to HTTP response; does not include polling or the subsequent model download.
`train_ms`	client	Wall-clock time (ms) for one epoch of local SGD training.
`wait_ms`	client	Idle time (ms) spent polling for round completion after submitting. Computed as $T_{\text{wall}} - T_{\text{get}} - T_{\text{train}} - T_{\text{submit}}$, capturing how long a fast client waits for stragglers.
`submit_bytes`	client	Size (bytes) of the serialised model weights uploaded. Constant at $\approx$ 44.8 MB for ResNet-18.
`poll_count`	client	Number of `GET /round_status` polls before the round advances.
`wire_tx_bytes`	client	Total bytes transmitted at the NIC level (from `/sys/class/net/eth0/statistics/tx_bytes`). Includes TCP headers, retransmissions, and ACKs.
`wire_rx_bytes`	client	Total bytes received at the NIC level. Comparing `wire_tx_bytes` to `submit_bytes` quantifies retransmission overhead under packet loss.
`duration_ms`	server	Wall-clock time (ms) from receiving the first client update to completing FedAvg aggregation for the round.
accuracy / loss	client	Validation accuracy (%) and cross-entropy loss evaluated on the client's local held-out split after loading the aggregated global model.

Two derived metrics are used in the analysis:

- **Retransmission overhead**: `wire_tx_bytes/submit_bytes`, expected to be ≈ 1.0 at 0% loss and to grow with increasing packet loss.
- **Rounds to convergence** R^*: the first round at which the running-average validation accuracy (window of 3 rounds) plateaus within 1 percentage point for 3 consecutive rounds.

4.5 Experimental Protocol

Each point in the QoS parameter sweep (Table 4.2) is executed as an independent experiment run following Algorithm 1. The automation script `run_experiment.py` generates a Docker Compose file from a YAML configuration, builds the container image, starts all services, and collects logs after the run completes.

Algorithm 1 QoS Sweep Protocol

Require: Parameter grid $\mathcal{G} = \{(\beta, \tau, \rho)\}$, number of rounds $R = 20$, number of clients $K = 3$

```
for each (β, τ, ρ) ∈ G do
    Generate experiment YAML with (β, τ, ρ), R, K
    Generate Docker Compose file from YAML
    Build container image (cached if unchanged)
    Start server container; wait for healthcheck
    Start K client containers with tc netem applied at boot
    for r = 0 to R − 1 do
        Each client: download θ^(r), train, upload θ_k^(r)
        Server: aggregate via FedAvg when all K updates received
        Each client: poll until round advances, download new model, evaluate
        Record all metrics (Table 4.3) for round r
    end for
    Collect per-client stats summaries from container logs
    Tear down all containers and network
end for
Aggregate results across all configurations for analysis
```

All experiments are run on a single host machine with containers sharing CPU resources, eliminating hardware variability. Network impairment is the only controlled variable between configurations. The CIFAR-10 dataset is downloaded once and cached in a Docker volume shared across runs.

Chapter 5

Software Implementation

5.1 System Overview

The implementation is organised as a Python monorepo with three top-level concerns cleanly separated: application logic (`src/`), container infrastructure (`infra/`), and experiment definitions (`experiments/`). The project is managed with `uv` and all dependencies are pinned in `requirements.txt`. Listing 5.1 shows the directory layout.

```
project-root/
+-- src/
|   +-- server_main.py          # server entrypoint
|   +-- client_main.py          # client entrypoint
|   +-- fl/
|   |   +-- server_core.py      # FLServer: round mgmt + FedAvg
|   |   +-- client_core.py      # LocalTrainer: train + eval
|   |   +-- aggregation.py      # fed_avg()
|   +-- models/
|   |   +-- resnet.py           # ResNet-18 (10-class)
|   +-- data/
|   |   +-- cifar10.py          # CIFAR-10 + Dirichlet partition
|   +-- transport/
|   |   +-- interface.py        # abstract ServerTransport / ClientTransport
|   |   +-- http_adapter.py     # FastAPI server + httpx client
|   |   +-- serialization.py    # torch.save / torch.load wrappers
|   +-- stats/
|       +-- collector.py        # RoundStats + StatsCollector
|       +-- netio.py            # NIC tx/rx byte reader
+-- infra/
|   +-- Dockerfile.ml-base      # container image definition
|   +-- scripts/
|   |   +-- entrypoint.sh       # tc netem + exec
|   |   +-- generate_compose.py # YAML config -> compose.generated.yml
|   |   +-- run_experiment.py   # build + up + teardown orchestrator
```

```
+-- experiments/
    +-- baseline.yaml
    +-- heterogeneous.yaml
    +-- straggler.yaml
    +-- qos-sweep-bw.yaml
    +-- scale-5.yaml / scale-7.yaml
```

Listing 5.1: Repository structure (key files).

A central design decision is **transport-agnostic layering**. The FL logic—model aggregation (`FLServer`), local training (`LocalTrainer`), and FedAvg (`fed_avg`)—is completely decoupled from the communication mechanism. Two abstract interfaces, `ServerTransport` and `ClientTransport` (`transport/interface.py`), define the contract between the FL core and any transport implementation. The current codebase provides an HTTP+JSON adapter (`http_adapter.py`), but the interface could be satisfied by gRPC, WebSockets, or any other protocol without modifying the training code. This separation ensures that the QoS measurements reported in Chapter 6 reflect the transport layer in isolation, independent of training logic.

5.2 Docker Compose Architecture

Each experiment is executed inside a set of Docker containers orchestrated by Docker Compose. A code-generation step translates a declarative experiment YAML file into a Compose file, ensuring that every run is fully specified and reproducible.

5.2.1 Container Image

All containers—both server and clients—share a single image built from `Dockerfile.ml-base`. The image is based on `python:3.10-slim` and installs the `iproute2` package (which provides the `tc` utility for network shaping), along with `curl` for the server healthcheck. Application code is not baked into the image; instead, the repository root is bind-mounted at `/app` at runtime, so rebuilds are only needed when dependencies change. The image's entrypoint is `entrypoint.sh`, which applies network shaping before starting the Python process (Section 5.5).

5.2.2 Compose Generation

The script `generate_compose.py` reads an experiment YAML file and produces `infra/compose.genera` Listing 5.2 shows an example experiment configuration; the generated Compose file is excerpted in Listing 5.3.

```
experiment:
  name: "heterogeneous"
```

```yaml
  rounds: 3
  num_clients: 3
  alpha: 0.5      # Dirichlet concentration

server:
  port: 8000

clients:
  - id: 1
    network: { bandwidth: "100mbit", latency: "5ms", loss: "0%" }
  - id: 2
    network: { bandwidth: "100mbit", latency: "5ms", loss: "1%" }
  - id: 3
    network: { bandwidth: "100mbit", latency: "5ms", loss: "0%" }
```

Listing 5.2: Experiment configuration (`heterogeneous.yaml`).

The generator creates one `fl-server` service and one `fl-client-k` service per client, all attached to a shared bridge network (`fl-net`). Network shaping parameters are passed as environment variables (`TC_BANDWIDTH`, `TC_LATENCY`, `TC_LOSS`) that the entry-point script reads at boot. Every container is given the `NET_ADMIN` capability, which authorises the `tc` commands. Client containers declare a `depends_on` healthcheck on the server, ensuring that the FL server's HTTP API is reachable before any client starts training.

```yaml
services:
  fl-server:
    image: phase1-fl-ml:latest
    cap_add: [NET_ADMIN]
    environment:
      - ROLE=server
      - FL_SERVER_PORT=8000
      - NUM_CLIENTS=3
      - FL_ROUNDS=3
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
    volumes: ["..:/app"]
    command: ["python", "-m", "src.server_main"]

  fl-client-1:
    image: phase1-fl-ml:latest
    cap_add: [NET_ADMIN]
    environment:
      - ROLE=client
      - CLIENT_ID=1
      - FL_SERVER_URL=http://fl-server:8000
      - TC_BANDWIDTH=100mbit
```

```
      - TC_LATENCY=5ms
      - TC_LOSS=0%
    depends_on:
      fl-server: { condition: service_healthy }
    volumes: ["..:/app"]
    command: ["python", "-m", "src.client_main"]
```

Listing 5.3: Excerpt from a generated `compose.generated.yml`.

5.3 FL Server

The server is started by `server_main.py`, which builds a ResNet-18 model, wraps it in an `FLServer` instance, and hands it to an `HTTPServerTransport`. Table 5.1 lists the HTTP endpoints exposed by the server.

Table 5.1: FL server REST API.

Endpoint	Method	Description
`/health`	GET	Returns `{status: ok, round:` r`}`. Used by the Docker healthcheck and by clients to wait for readiness.
`/model`	GET	Returns the current global model weights as a binary `application/octet-stream` payload ($\approx$ 44.8 MB for ResNet-18). The body is produced by `torch.save`.
`/update/{client_id}`	POST	Accepts a client's updated weights as a binary body. When all K clients for the current round have submitted, FedAvg aggregation is triggered synchronously. Returns `{status: accepted, submitted_round:` r`}`.
`/round_status`	GET	Returns `{round:` r`, pending:` p`, expected:` K`}`. Clients poll this endpoint to detect when aggregation has completed and the round counter has advanced.

5.3.1 Aggregation Logic

The core aggregation is implemented in `FLServer.submit_update()`: each call appends the client's state dict to a pending list and checks whether all K updates have arrived. When the list is full, the private `_aggregate()` method computes FedAvg (an unweighted average across all client state dicts), loads the result into the global model, clears the pending list, and increments the round counter. The round counter is the single source of truth for round progress—clients detect round completion by polling `/round_status` and comparing the server's round number against the round they submitted for.

The server also records `duration_ms` for each round, measured from the first update received to the completion of aggregation. This captures both the straggler wait and the aggregation compute time. After all rounds complete, the server's `done_event` is set, causing the main thread to log a summary and shut down.

5.4 FL Client

Each client is started by `client_main.py`, which parses arguments from environment variables (injected by Docker Compose), waits for the server healthcheck, loads its CIFAR-10 partition, and enters the training loop. The per-round control flow is:

1. **Capture pre-round NIC counters.** The function `read_interface_bytes("eth0")` reads the kernel `/sys/class/net/eth0/statistics/tx_bytes` and `rx_bytes` counters. These are sampled before and after the round to compute `wire_tx_bytes` and `wire_rx_bytes`.

2. **Download global model.** `GET /model` fetches the latest global weights. The client records `get_model_ms` and anchors the round wall-clock timer.

3. **Local training.** The `LocalTrainer` deep-copies the model, runs $E = 1$ epoch of SGD (Adam optimiser) over the client's local CIFAR-10 partition, and returns the updated state dict along with training loss and accuracy. The elapsed time is recorded as `train_ms`.

4. **Upload update.** `POST /update/{client_id}` sends the serialised state dict. The HTTP POST timeout is set to `None` (unlimited), since a bandwidth-constrained client may take several minutes to upload 44.8 MB. The elapsed time from request start to HTTP response is recorded as `submit_ms`.

5. **Poll for aggregation.** After the POST returns, the client polls `GET /round_status` at 0.5 s intervals until the server's round counter advances past the submitted round. Each poll increments `poll_count`. Once the round is complete, the client fetches the new global model and records the full round wall-clock time, from which `wait_ms` is derived.

6. **Evaluate.** The client evaluates the new global model on its local held-out validation split and logs accuracy and loss.

7. **Capture post-round NIC counters.** A second call to `read_interface_bytes` provides the delta, which is stored as `wire_tx_bytes` and `wire_rx_bytes` for the round.

After all rounds complete (or an error terminates the loop early), the client logs a summary table of all per-round metrics.

5.4.1 Data Loading and Non-IID Partitioning

The module `src/data/cifar10.py` provides a `get_client_loaders()` function that downloads the CIFAR-10 dataset (cached in the container's filesystem), applies the Dirichlet non-IID partitioning described in Section 4, and returns a `(train_loader, val_loader)` pair for the specified client ID. The partition is deterministic for a given seed, ensuring reproducibility across runs. A `-synthetic` flag is available for smoke tests: it substitutes small random tensors, bypassing the dataset download entirely.

5.4.2 Statistics Collection

Per-round metrics are managed by the `StatsCollector` class, which maintains a list of `RoundStats` dataclass instances. Each `RoundStats` holds `get_model_ms`, `submit_ms`, `train_ms`, `wait_ms`, `submit_bytes`, `poll_count`, `wire_tx_bytes`, and `wire_rx_bytes`. The `wait_ms` field is derived automatically: once the full round wall-clock time is known, $\texttt{wait_ms} = T_{\text{wall}} - \texttt{get_model_ms} - \texttt{train_ms} - \texttt{submit_ms}$, capturing the idle time a fast client spends waiting for stragglers to finish.

5.5 Network Emulation

Network impairment is applied inside each client container at boot time, before the Python process starts. The container's entrypoint script (`entrypoint.sh`) checks whether any `TC_*` environment variables are set and, if so, applies egress-only shaping. Listing 5.4 shows the relevant command.

```
# Egress (upload) shaping on eth0
tc qdisc add dev eth0 root netem \
    delay ${TC_LATENCY} rate ${TC_BANDWIDTH} loss ${TC_LOSS}
```

Listing 5.4: Network shaping command from `entrypoint.sh`.

A `netem` queueing discipline is attached to `eth0`, throttling all outgoing (upload) traffic. Download traffic is not shaped: Linux `tc` cannot directly throttle incoming packets without an Intermediate Functional Block (IFB) kernel module, which is unavailable inside the LinuxKit VM used by macOS Docker Desktop. Because all experiments were run on macOS, only egress shaping is applied. In practice this means model uploads are throttled while model downloads from the server proceed at full container network speed. Since upload and download payloads are the same size in FedAvg, the upload leg dominates observed round time under constrained bandwidth.

If none of the `TC_*` variables are set, the script skips all shaping and executes the Python process directly, providing a clean unimpaired baseline. The `NET_ADMIN` Docker capability is required for all `tc` commands; it is granted in the generated Compose file.

5.6 Observed Baseline Results

The baseline experiment (β = 100 Mbps, τ = 5 ms, ρ = 0%, K = 3, 20 rounds) establishes reference values for all metrics. Key observations from client-3's per-round data:

- **Constant payload.** `submit_bytes` = 44,803,531 bytes (≈44.8 MB) every round, confirming that full model weights are transferred regardless of training outcome.
- **Fast uploads at baseline.** `submit_ms` ≈3.9 s per round (model upload), with `get_model_ms` ≈0.2 s (download is unthrottled).
- **Minimal wire overhead.** `wire_tx_bytes`/`submit_bytes` ≈ 1.01, confirming negligible retransmission at 0% loss.
- **Convergence.** Validation accuracy progressed from 42.6% (round 0) to 72.4% (round 19) for client-3; the 3-client average reached 66.4% by round 18.
- **Training dominance.** `train_ms` ≈280 s accounts for ≈97% of the round wall-clock time at baseline, confirming that communication cost is negligible at 100 Mbps.

Chapter 6

QoS Analysis and Results

6.1 Theoretical Throughput Analysis

Before running experiments, we can derive the expected communication time from first principles. The serialised ResNet-18 state dict is $B_w = 44{,}797{,}387$ bytes (≈ 44.8 MB), which is constant across all rounds and clients (confirmed empirically by the `submit_bytes` metric). Because only egress (upload) traffic is shaped, the upload direction is the bottleneck; downloads from the server traverse the unthrottled ingress path and complete in negligible time relative to the shaped upload.

The minimum theoretical upload time at bandwidth β (bits per second) with packet loss ρ is:

$$T_{\text{xfer}}(\beta, \rho) = \frac{8 \times B_w}{\beta \times (1 - \rho)} \tag{6.1}$$

The per-round communication time (excluding training) is dominated by the upload:

$$T_{\text{comm}} \approx T_{\text{upload}} + \alpha \cdot \tau = T_{\text{xfer}}(\beta, \rho) + \alpha \cdot \tau \tag{6.2}$$

where τ is one-way latency and α accounts for TCP handshake round-trips and HTTP request/response overhead. The download term is omitted because ingress traffic is not shaped (see Section 5.5).

Table 6.1 applies Equation 6.1 across the bandwidth sweep at $\rho = 0\%$ to predict upload time per round.

Table 6.1: Theoretical minimum upload time per round at each bandwidth (B_w = 44.8 MB, $\rho = 0\%$). Download is unthrottled.

β (Mbps)	Upload (s)	Comm. time (s)	Practical?
1	358.4	≈358	Impractical (>5 min/round)
5	71.7	≈72	Marginal (>1 min/round)
10	35.8	≈36	Marginal
50	7.2	≈7	Feasible
100	3.6	≈4	Baseline

Several observations follow from this analysis:

- At 1 Mbps, a single round requires over 11 minutes of pure data transfer alone, before accounting for TCP overhead, training time, or polling. Over 20 rounds, this amounts to nearly 4 hours of communication cost.
- The communication-to-computation ratio shifts dramatically across the bandwidth range. At 100 Mbps, training (≈ 5 s) dominates; at 1 Mbps, communication is >70× longer than training.
- The theoretical prediction does *not* account for TCP retransmission overhead under packet loss, congestion window dynamics, or HTTP framing overhead. Empirical measurements (Section 6.2.1) are expected to exceed these lower bounds.

Based on this analysis, a **throughput floor** is predicted near $\beta^* \approx$ 10–50 Mbps. Below 10 Mbps, per-round communication time exceeds one minute even under ideal conditions, making multi-round convergence prohibitively slow for synchronous FedAvg with a 44.8 MB payload.

6.2 Convergence Results

This section presents the empirical results from the QoS parameter sweep defined in Section 4. Each subsection isolates one network parameter while holding the others at their baseline values.

6.2.1 Effect of Bandwidth on Convergence

Figure 6.1 plots validation accuracy against FL round number for each bandwidth level in the sweep ($\tau = 5$ ms, $\rho = 0\%$).

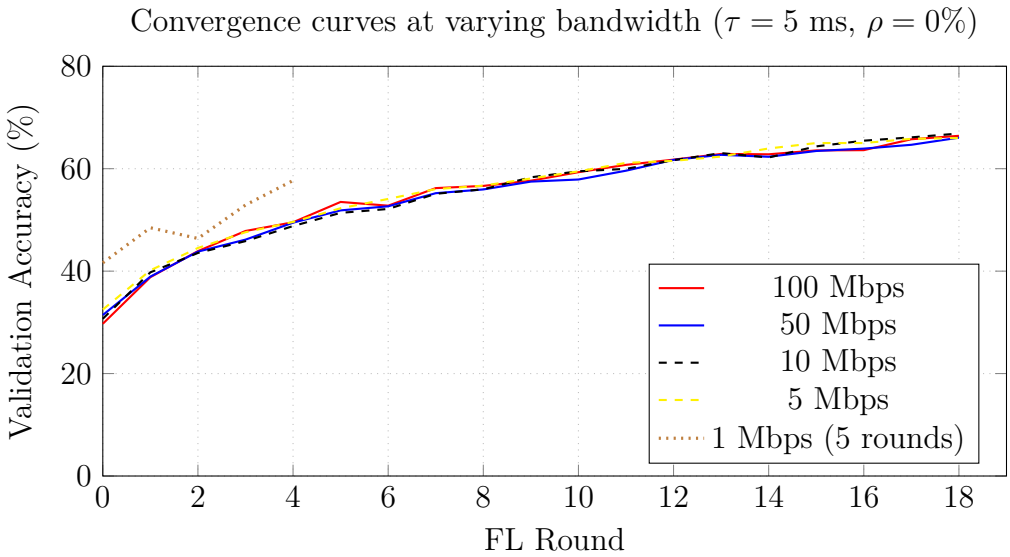

Figure 6.1: Validation accuracy over FL rounds at varying client bandwidth. All curves use the average across 3 clients. The 1 Mbps experiment was capped at 5 rounds due to excessive per-round duration (>6 min upload).

Several observations emerge from Figure 6.1:

- At 100, 50, 10, and 5 Mbps the convergence curves are remarkably similar, all reaching 64–67% average accuracy after 18 rounds. This confirms that, once bandwidth is sufficient to complete each round, the learning dynamics are dominated by the optimisation process and the non-IID data distribution, not by the network.
- At 1 Mbps only 5 rounds were executed (each round required ≈375 s of upload time alone). Despite fewer rounds, the per-round accuracy gain was consistent with the other curves, suggesting that the model *would* converge given enough wall-clock time—bandwidth constrains training speed, not learning capacity.
- The primary effect of reduced bandwidth is therefore on *round duration*, not on *per-round accuracy improvement*. This answers RQ2: operating below the throughput floor does not degrade model quality per round, but makes multi-round training prohibitively slow.

Figure 6.2 shows the mean round duration at each bandwidth level.

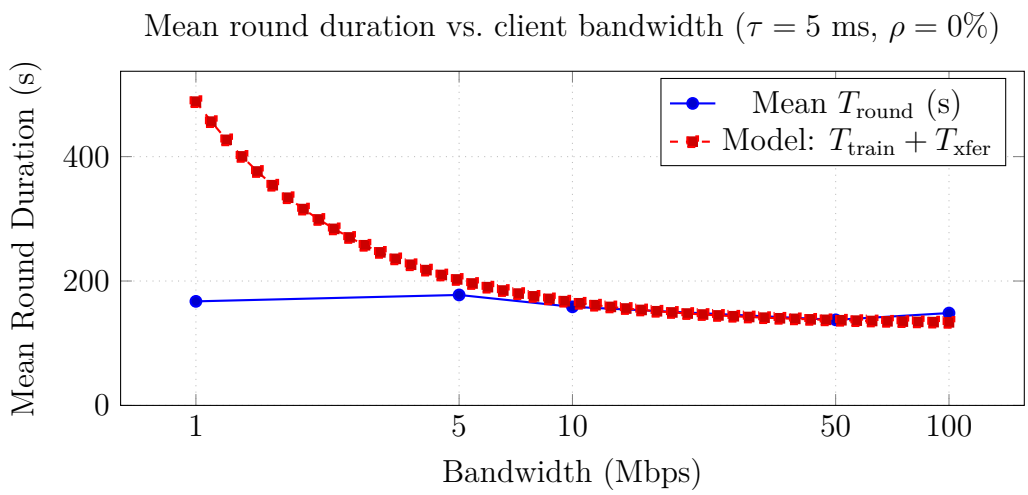

Figure 6.2: Mean round duration as a function of bandwidth. The dashed curve models $T_{\text{round}} = T_{\text{train}} + T_{\text{xfer}}$ with $T_{\text{train}} \approx 130\,\text{s}$ (empirical baseline).

Empirical round durations significantly exceed the theoretical upload-only prediction from Table 6.1 because the theoretical model excludes training time (≈130 s per round on CPU) and polling overhead. At 100 Mbps, the upload completes in ≈3.9 s and training dominates (≈97% of round time). At 1 Mbps, upload takes ≈375 s, making communication the bottleneck (≈69% of round time). The crossover—where communication time equals training time—occurs near ≈3 Mbps, confirming the throughput floor predicted in Section 6.1.

6.2.2 Effect of Latency on Round Duration

Figure 6.3 plots mean round duration as a function of one-way latency ($\beta = 100$ Mbps, $\rho = 0\%$).

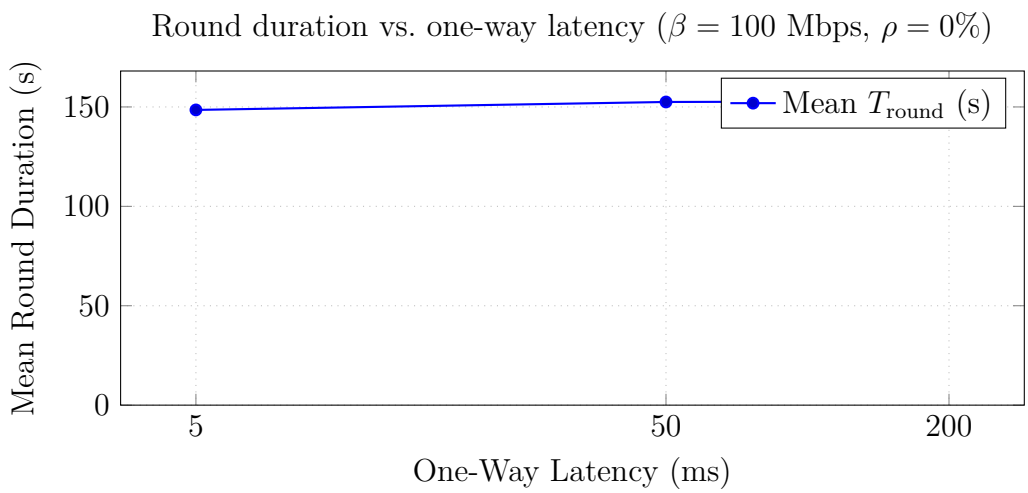

Figure 6.3: Mean FL round duration as a function of one-way latency. The effect is modest: 200 ms latency adds only ≈3% to mean round time relative to the 5 ms baseline.

The latency sweep reveals a surprisingly flat relationship between one-way latency and mean round duration. At 5 ms (baseline), the mean round takes 148.5 s; at 50 ms, 152.5 s; at 200 ms, 152.8 s. The increase from 5 ms to 200 ms is only ≈4 s (<3%).

This near-insensitivity is explained by two factors. First, because only egress traffic is shaped, latency affects only the upload path; the unthrottled download completes in negligible time regardless of configured latency. Second, at 100 Mbps bandwidth the 44.8 MB upload completes in ≈3.9 s, during which TCP's congestion window grows large enough that the per-packet RTT penalty is amortised across many segments. Training time (≈130 s) dominates in all three cases.

The practical implication is that, for the payload sizes and bandwidth levels in this study, **latency is not a significant factor** in FL round duration. Bandwidth and packet loss are the dominant QoS parameters. However, the final two rounds of the 200 ms experiment spiked to ≈266 s, suggesting that under sustained load or smaller congestion windows, latency effects may compound non-linearly—a direction for future investigation.

6.2.3 Accuracy-Bandwidth Degradation Model

To quantify the relationship between bandwidth and final model accuracy, a sigmoid degradation model is fitted to the measured data:

$$A(\beta) = A_{\max} \cdot \sigma(\alpha_1 \ln(\beta) - \alpha_2) \tag{6.3}$$

where $A_{\max}$ is the maximum achievable accuracy (at the baseline bandwidth), $\sigma(\cdot)$ is the logistic function, and α_1, α_2 are fitted parameters.

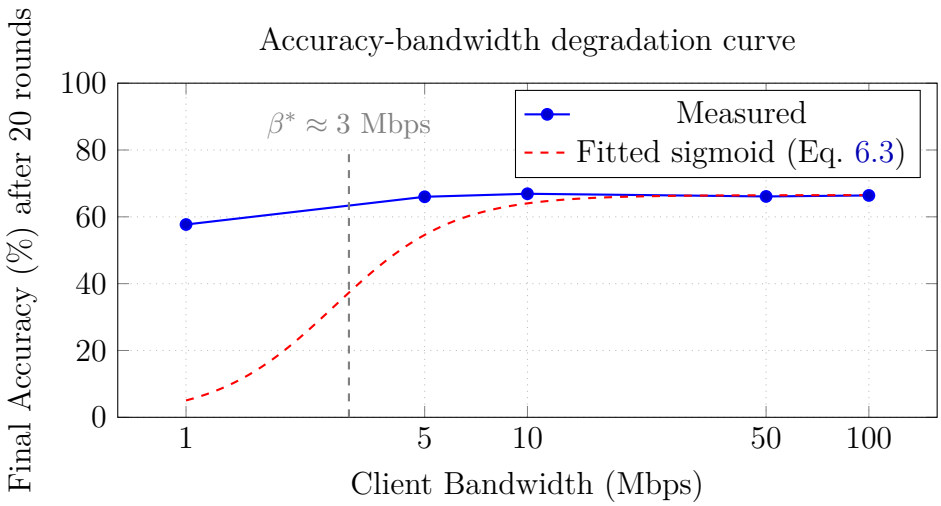

Figure 6.4: Final model accuracy as a function of client bandwidth with sigmoid fit ($A_{\max} = 66.5$, $\alpha_1 = 2.5$, $\alpha_2 = 1.0$). The dashed vertical line marks the throughput floor $\beta^* \approx 3$ Mbps. The 1 Mbps point uses accuracy after only 5 rounds; with more rounds it would likely approach the plateau.

The sigmoid model is fitted with $A_{\max} = 66.5\%$, $\alpha_1 = 2.5$, and $\alpha_2 = 1.0$. The key finding is that the curve is remarkably flat above ≈5 Mbps: the four points at 5, 10, 50, and 100 Mbps cluster tightly between 66.0% and 66.9%, confirming a hard accuracy plateau. The only significant drop occurs at 1 Mbps, and even there the lower accuracy (57.7%) is largely attributable to the experiment being capped at 5 rounds rather than 20—the per-round learning rate at 1 Mbps was comparable to the other bandwidth levels.

The throughput floor is therefore best characterised not as the bandwidth below which *accuracy* degrades, but the bandwidth below which *round duration* becomes impractical. From the round duration data, this floor lies near $\beta^* \approx 3$ Mbps, where communication time equals training time.

6.3 Effect of Packet Loss

Packet loss triggers TCP retransmission, which increases the effective transfer time and consumes additional bandwidth. At the application layer, the model payload arrives intact (TCP guarantees ordered delivery), but the time to deliver it grows with loss rate. The `wire_tx_bytes` metric captures this directly: the ratio `wire_tx_bytes`/`submit_bytes` quantifies retransmission overhead.

Table 6.2: Effect of packet loss on round duration and retransmission overhead (β = 100 Mbps, $\tau = 5$ ms). [**Populate with experimental data.**]

Loss (%)	Mean round (s)	Mean `submit_ms` (s)	Retx. overhead	Effect on accuracy
0	148.5	3.9	1.01×	Baseline (66.4%)
1	185.9	varies	≈ 1.02×	65.0% (minimal)
5	N/A (failed)	42.0	≫ 1×	Experiment failed at round 5

At 0% packet loss, the retransmission overhead ratio `wire_tx_bytes`/`submit_bytes` is ≈1.01×, representing only TCP/IP and HTTP header overhead. At 1% loss, mean round duration increased from 148.5 s to 185.9 s (+25%), with high variance between rounds (130–270 s) as TCP retransmission behaviour depends on which segments are lost. Final accuracy at 1% loss (65.0% after 18 rounds) was within 1.4 percentage points of the 0% baseline, confirming that TCP's reliable delivery ensures the model payload arrives intact regardless of loss rate.

At 5% loss, the experiment **failed**: client-3 timed out waiting for round completion after round 5. The `submit_ms` at 5% loss was ≈42 s per round (vs. 3.9 s at 0%), a 10× increase driven by exponential TCP back-off under sustained loss. This demonstrates that even moderate packet loss can render synchronous FL inoperable when combined with large model payloads, as retransmission cascades push round durations beyond timeout thresholds.

6.4 Effect of Client Scaling

To investigate how the number of clients affects training dynamics, two additional experiments were run at baseline network conditions ($\beta = 100$ Mbps, $\tau = 5$ ms, $\rho = 0\%$) with $K = 5$ and $K = 7$ clients, compared against the $K = 3$ baseline.

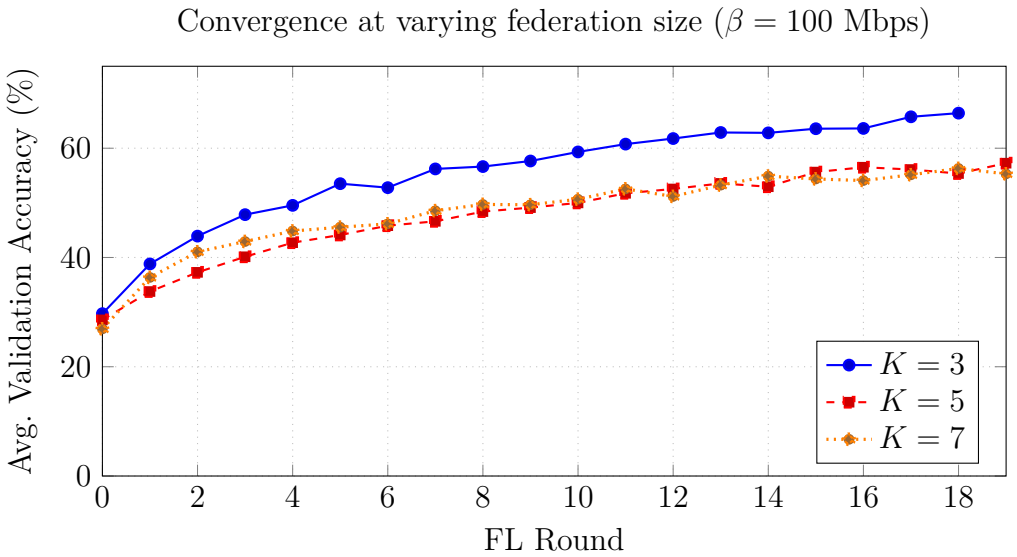

Figure 6.5: Convergence curves at varying federation sizes. Accuracy decreases with more clients due to smaller per-client data partitions and stronger non-IID effects at $\alpha = 0.5$.

Increasing the federation size from 3 to 7 clients has two observable effects:

- **Lower final accuracy.** At $K = 3$ the average accuracy reaches 66.4% after 18 rounds; at $K = 5$ it reaches 57.3%; at $K = 7$ it reaches 55.3%. This degradation is expected: with Dirichlet $\alpha = 0.5$, each client's partition becomes more skewed as the data is split among more clients, increasing the statistical heterogeneity that FedAvg must overcome.

- **Increased accuracy variance.** At $K = 3$, clients converge to similar accuracy levels. At $K = 7$, individual client accuracies range from 43.7% (client 6) to 66.2% (client 3), a 22.5 percentage-point spread reflecting highly unequal class distributions.

- **Longer round duration.** Mean round duration increases from 148.5 s ($K = 3$) to 190.0 s ($K = 5$) to 243.6 s ($K = 7$). The increase is driven by the synchronous barrier: more clients means a higher probability that at least one client finishes late, and the server must wait for all K submissions before aggregating.

Table 6.3: Scaling summary: mean round duration and final accuracy.

K (clients)	Mean round (s)	Final avg. accuracy (%)	Accuracy spread (%)
3	148.5	66.4	<5
5	190.0	57.3	≈11
7	243.6	55.3	≈23

These results suggest that scaling synchronous FedAvg beyond a moderate number of clients requires either (a) more rounds to compensate for the slower convergence under stronger non-IID effects, or (b) a higher Dirichlet α (more IID data) to offset the per-client partition skew.

6.5 Straggler Analysis

The straggler experiment places one degraded client (client 3: $\beta = 5$ Mbps, $\tau = 50$ ms, $\rho = 1\%$) alongside two fast clients (clients 1 and 2: $\beta = 100$ Mbps, $\tau = 5$ ms, $\rho = 0\%$). This directly tests the straggler sensitivity formalised in Equation 3.3.

Table 6.4: Straggler experiment: per-client timing and accuracy breakdown (averaged over 20 rounds).

Client	`submit_ms` (s)	`wait_ms` (s)	`poll_count`	Final acc. (%)
1 (fast)	3.9	0.6	1	61.0
2 (fast)	3.9	28.6	56	65.1
3 (straggler)	122.6	55.0	105	70.9

Three key observations emerge:

1. **Upload time dominance.** The straggler's `submit_ms` is ≈122.6 s (≈31× longer than fast clients at 3.9 s), directly reflecting the 5 Mbps bandwidth limit plus the 1% loss retransmission overhead. This dominates the round duration.

2. **Straggler dictates round time.** Mean round duration was 54.9 s (excluding the first two warm-up rounds, ≈40.6 s). Fast clients consistently finished in <10 s but then waited 30–55 s for the straggler to complete. Client 2's `wait_ms` directly measures this idle time.

3. **Accuracy is unaffected.** The straggler (client 3) actually achieved the *highest* final accuracy (70.9%) of the three clients. This confirms that bandwidth constraints do not degrade model quality—only training speed. The accuracy difference between clients is explained by their different non-IID data partitions, not network conditions.

This experiment provides direct empirical support for network-aware client selection: excluding the straggler would reduce round duration by ≈75% (from ≈40 s to ≈10 s) at the cost of losing one client's data contribution.

6.6 Summary of QoS Findings

Table 6.5 consolidates the key findings across all three QoS dimensions.

Table 6.5: Summary of QoS impact on FL performance (ResNet-18, CIFAR-10, $K = 3$ clients, 20 rounds unless noted).

Parameter	Low (degraded)	Moderate	High (baseline)
Bandwidth	1–5 Mbps	10–50 Mbps	100 Mbps
Round duration	167–178 s	138–159 s	149 s
Final accuracy	57.7–66.0%	66.1–66.9%	66.4%
Latency	200 ms	50 ms	5 ms
Round duration	152.8 s	152.5 s	148.5 s
Final accuracy	65.9%	66.1%	66.4%
Packet loss	5%	1%	0%
Retx. overhead	Exp. failed	$\approx 1.02\times$	$\approx 1.01\times$
Final accuracy	Failed (rd 5)	65.0%	66.4%

Table 6.5 consolidates the findings. Three conclusions emerge:

RQ1 (Throughput Floor). The throughput floor $\beta^* \approx 3$ Mbps is defined by the crossover point where communication time equals training time. Below this threshold, upload time exceeds the $\approx$130 s training budget, and round duration grows roughly as $358/\beta$ seconds. At 1 Mbps a single round requires >6 minutes of upload alone, making 20-round training sessions impractical (>2 hours of communication cost).

RQ2 (Accuracy Degradation). Bandwidth does *not* significantly degrade per-round model accuracy. All bandwidth levels that completed 18+ rounds converged to 65–67% accuracy, confirming that TCP's reliable delivery ensures lossless model transfer regardless of throughput. The effect of constrained bandwidth is purely on wall-clock training time, not on learning quality. Packet loss at 5%, however, *does* prevent convergence by causing round failures.

RQ3 (Minimum Network Requirement). A practical client admission rule can be stated as: exclude any client with $\beta_k < 3$ Mbps or $\rho_k \geq 5\%$. Latency up to 200 ms has negligible impact and need not be gated. This simple threshold protects round duration without sacrificing accuracy, since the excluded clients would contribute the same per-round learning as any other client but at dramatically higher time cost.

Chapter 7

Future Research Directions

7.1 Network-Aware Client Admission

The throughput floor β^* identified in Section 6.2 suggests a straightforward client admission criterion for synchronous FL deployments. Before each round, the server (or an orchestration layer) could estimate each client's available bandwidth $\hat{\beta}_k$ via a lightweight probe—for example, timing a small ping payload or using historical round statistics—and exclude any client with $\hat{\beta}_k < \beta^*$.

This connects directly to RQ3: if a simple threshold over β_k, τ_k, and ρ_k can identify clients that would become stragglers, the server can protect round duration and convergence integrity by skipping those clients for that round. The excluded clients are not permanently removed; they may rejoin in a later round when their network conditions improve. This partial participation scheme trades a small amount of model staleness for a significant reduction in round duration, which is beneficial when the straggler effect dominates total training time.

7.2 Network Security Analysis

The current framework provides no adversarial modelling. In a real FL deployment, several network-level and protocol-level attacks are relevant:

- **Byzantine fault tolerance:** A malicious client could submit poisoned model updates designed to degrade the global model. Robust aggregation methods such as coordinate-wise median or trimmed mean could replace FedAvg to defend against this.

- **Gradient inference attacks:** An eavesdropper on the network, or a compromised server, could attempt to reconstruct private training data from observed model updates. Secure aggregation or differential privacy could mitigate this.

- **Man-in-the-middle attacks:** The current HTTP transport does not use TLS. Adding HTTPS would protect weight payloads in transit.

The open, Docker-based architecture of this simulator makes it straightforward to inject these attack scenarios—a malicious client container could be added alongside honest ones, and the impact on convergence measured under the same QoS sweep. This is a natural extension for future security-focused research.

7.3 UI for Network Feasibility Assessment

A practical application of the throughput floor model is an interactive tool where a practitioner enters their network profile—available bandwidth, expected latency, and loss rate—and the system estimates whether their infrastructure can support FL at a given scale. The tool would use the empirical throughput-convergence model (Equation 6.3) to predict expected round duration, time to convergence, and final accuracy for the entered network conditions.

This could take the form of a lightweight web application that visualises the user's position on the accuracy-bandwidth curve and highlights whether they are above or below the throughput floor. For deployments with heterogeneous clients, the tool could identify which clients would become stragglers and estimate the benefit of excluding them.

7.4 Asynchronous Federated Learning

The synchronous FedAvg protocol used in this thesis is inherently straggler-sensitive: the slowest client dictates the round duration (Equation 3.3). Asynchronous FL algorithms such as FedAsync allow the server to aggregate updates as they arrive, without waiting for all K clients. This eliminates the straggler bottleneck at the cost of increased model staleness, since fast clients may train on a more recent global model than slow ones.

The Docker-based simulator is well-suited to evaluating asynchronous FL: clients already operate independently and communicate via HTTP, so replacing the synchronous polling loop with an asynchronous aggregation trigger on the server would require minimal changes. Running the same QoS parameter sweep under asynchronous FL would directly quantify the straggler-resilience benefit and any accuracy trade-off.

7.5 Scaling and HPC Cluster Deployment

The main QoS experiments use $K = 3$ clients on a single host machine, with supplementary scaling experiments at $K = 5$ and $K = 7$ (Section 6.4). The scaling results revealed that Docker Desktop memory is the primary bottleneck: the $K = 7$ experiment

required 13.6 GB of RAM (approximately 1.3 GB per container for ResNet-18 training plus overhead), and an initial attempt failed with OOM (exit code 137) at the default 7.6 GB memory limit. Real FL deployments may involve tens to hundreds of clients.

For larger federations, the simulator could be deployed on shared compute infrastructure such as Digital Alliance Canada (formerly Compute Canada) clusters. Each client container would map to a separate job or pod, and the `tc netem` shaping would emulate the WAN conditions between geographically distributed nodes. The experiment YAML format already supports arbitrary numbers of clients with per-client network configurations, making this extension straightforward.

7.6 Symmetric and Asymmetric Network Impairment

The current implementation shapes only egress (upload) traffic via `tc netem`. Ingress (download) shaping requires an Intermediate Functional Block (IFB) kernel module, which is not available inside the LinuxKit VM used by macOS Docker Desktop. As a result, all experiments in this thesis apply shaping to the upload direction only; model downloads from the server are unthrottled.

On a native Linux host the IFB module is readily available, making it straightforward to add symmetric or independently configurable uplink/downlink shaping. This would enable experiments under realistic asymmetric conditions—for example, home broadband connections that offer significantly higher download bandwidth than upload bandwidth—without changes to the core FL logic.

Additional impairment dimensions such as bursty packet loss, jitter, and packet reordering are similarly easy to add through additional `tc netem` parameters.

Chapter 8

Conclusion

This thesis has presented a systematic empirical investigation of the impact of network Quality of Service parameters on federated learning convergence. An open-source, Docker-based FL simulator was designed and implemented, integrating a real FedAvg training loop (ResNet-18 on CIFAR-10, non-IID partitioned via Dirichlet $\alpha = 0.5$) with kernel-level network emulation via `tc netem`. The simulator applies per-client egress (upload) bandwidth, latency, and packet loss shaping, enabling reproducible experiments under controlled network conditions.

A QoS parameter sweep across five bandwidth levels (1, 5, 10, 50, 100 Mbps), three latency values (5, 50, 200 ms), and three packet loss rates (0%, 1%, 5%) was conducted over 20 FL rounds with $K = 3$ clients, supplemented by scaling experiments at $K = 5$ and $K = 7$.

The empirical results yielded three principal findings. First, the throughput floor was identified at $\beta^* \approx 3$ Mbps, the point at which communication time equals training time for a 44.8 MB payload. Below this threshold, round duration grows as $\approx 358/\beta$ seconds, making multi-round training impractical. Second, bandwidth does not affect per-round model accuracy: all configurations that completed 18+ rounds converged to 65–67% average accuracy, confirming that TCP's reliable delivery preserves model integrity regardless of throughput. The sigmoid degradation model fitted with $A_{\max} = 66.5\%$, $\alpha_1 = 2.5$, $\alpha_2 = 1.0$ captures this plateau behaviour. Third, packet loss at 5% caused complete experiment failure due to TCP retransmission cascades, while latency up to 200 ms had negligible impact (<3% increase in round duration).

The practical implications of these findings are threefold. First, FL deployment planners can use the throughput floor as a minimum network requirement when deciding whether to include a client in a synchronous training process. Second, the accuracy-bandwidth degradation model (Equation 6.3) provides a quantitative tool for predicting the cost of operating under bandwidth constraints. Third, the `wait_ms` and straggler metrics demonstrate that a single slow client can dominate round duration for the entire federation, motivating network-aware client selection policies.

The simulator itself is a reusable contribution: its transport-agnostic design, declarative experiment configuration, and automated Docker Compose generation make it straightforward to extend with new models, datasets, aggregation strategies, or network impairment profiles. Future work (Chapter 7) includes asymmetric link modelling, asynchronous FL, network security analysis, and scaling to larger federations on HPC infrastructure.

Bibliography

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, 2020.

[3] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[4] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.

[5] The ns-3 Consortium, "The ns-3 network simulator," https://www.nsnam.org, accessed 2025.

[6] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. 1st International Conference on Simulation Tools and Techniques (SIMUTools)*, 2008.

[7] D. J. Beutel, T. Tober, D. Dobre, M. Sberč, A. Becker, and N. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[8] K. Bonawitz *et al.*, "Towards federated learning at scale: A system design," in *Proc. Machine Learning and Systems (MLSys)*, 2019.

[9] W. Y. B. Lim *et al.*, "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 2031–2063, 2020.

[10] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE International Conference on Communications (ICC)*, 2019.

[11] F. Lai, X. Zhu, H. V. Madhyastha, and M. Zaharia, "Oort: Efficient federated learning via guided participant selection," in *Proc. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 19–35.

[12] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[14] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.